

** Machine Learning Model for Medical Insurance Cost Prediction **

Imports

```
!pip install seaborn

# Imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

from sklearn.metrics import mean_squared_error, r2_score

sns.set(style="darkgrid")
```

Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
 Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.0.2)
 Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.2.2)
 Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-packages (from seaborn) (3.10.0)
 Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.1.1)
 Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.53.0)
 Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.5)
 Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)
 Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
 Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)
 Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2025.2)
 Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2026.1)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

Load Dataset

```
df=pd.read_csv("/content/Insurance.csv")
df.head()
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Basic Exploration

```
print("Shape:", df.shape)
print("\nColumns:\n", df.columns)
print("\nInfo:\n")
df.info()

print("\nSummary:\n", df.describe())
```

Shape: (1338, 7)

Columns:

Index(['age', 'sex', 'bmi', 'children', 'smoker', 'region', 'charges'], dtype='object')

Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1338 non-null   int64
```

```
1 sex      1338 non-null object
2 bmi      1338 non-null float64
3 children 1338 non-null int64
4 smoker   1338 non-null object
5 region   1338 non-null object
6 charges  1338 non-null float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

Summary:

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

Check Missing Values

```
df.isnull().sum()
```

```
0
age      0
sex      0
bmi      0
children 0
smoker   0
region   0
charges  0

dtype: int64
```

Feature Engineering (Advanced)

```
# Create BMI Category (important feature)
df['bmi_category'] = pd.cut(df['bmi'],
                             bins=[0,18.5,25,30,100],
                             labels=['Underweight','Normal','Overweight','Obese']).astype(str)

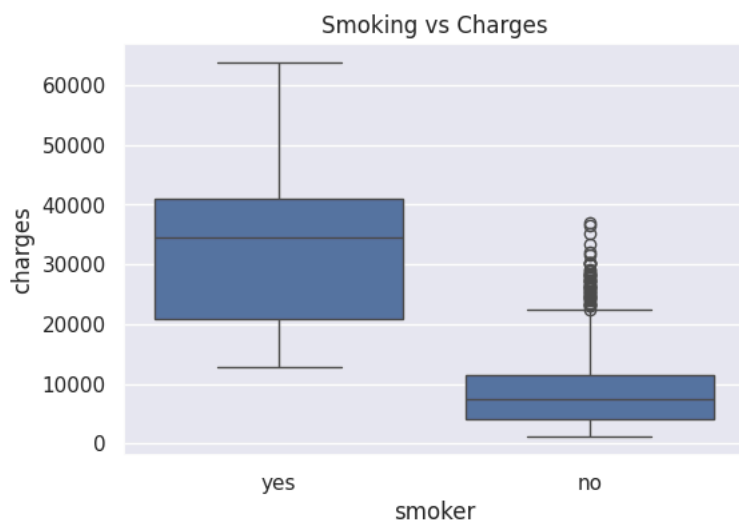
# Interaction feature
df['age_bmi'] = df['age'] * df['bmi']

df.head()
```

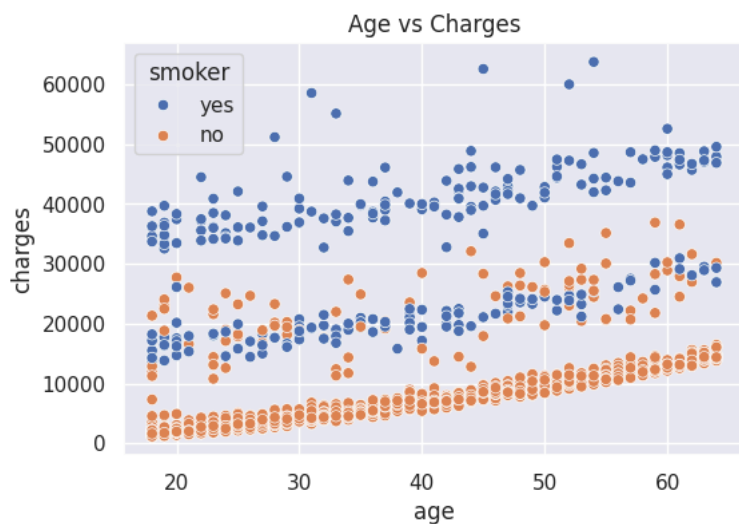
	age	sex	bmi	children	smoker	region	charges	bmi_category	age_bmi
0	19	female	27.900	0	yes	southwest	16884.92400	Overweight	530.100
1	18	male	33.770	1	no	southeast	1725.55230	Obese	607.860
2	28	male	33.000	3	no	southeast	4449.46200	Obese	924.000
3	33	male	22.705	0	no	northwest	21984.47061	Normal	749.265
4	32	male	28.880	0	no	northwest	3866.85520	Overweight	924.160

Advanced Visualizations

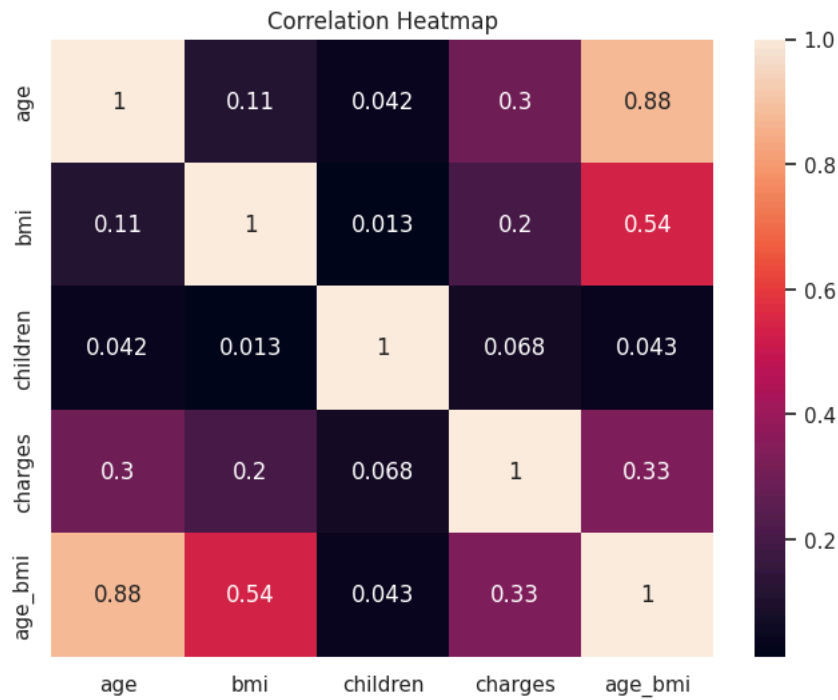
```
plt.figure(figsize=(6,4))
sns.boxplot(x='smoker', y='charges', data=df)
plt.title("Smoking vs Charges")
plt.show()
```



```
plt.figure(figsize=(6,4))
sns.scatterplot(x='age', y='charges', hue='smoker', data=df)
plt.title("Age vs Charges")
plt.show()
```



```
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.title("Correlation Heatmap")
plt.show()
```



Define Features & Target

```
X = df.drop('charges', axis=1)
y = df['charges']
```

Preprocessing Pipeline

```
numerical_features = ['age', 'bmi', 'children', 'age_bmi']
categorical_features = ['sex', 'smoker', 'region', 'bmi_category']

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numerical_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', drop='first'), categorical_features)
])
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Build Models (Advanced)

```
models = {
    "Linear Regression": LinearRegression(),
    "Ridge": Ridge(),
    "Random Forest": RandomForestRegressor(),
    "Gradient Boosting": GradientBoostingRegressor()
}

results = {}
```

Train & Evaluate

```
for name, model in models.items():
    pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    pipeline.fit(X_train, y_train)
    preds = pipeline.predict(X_test)

    r2 = r2_score(y_test, preds)
```

```

mse = mean_squared_error(y_test, preds)

results[name] = r2

print(f"{name}")
print("R2 Score:", r2)
print("MSE:", mse)
print("-----")

```

```

Linear Regression
R2 Score: 0.7806664175447021
MSE: 34051260.449731305
-----
Ridge
R2 Score: 0.780810976408679
MSE: 34028817.86947299
-----
Random Forest
R2 Score: 0.8721073039001602
MSE: 19855178.836563013
-----
Gradient Boosting
R2 Score: 0.8830384228129573
MSE: 18158136.491565533
-----

```

Best Model Selection

```

best_model_name = max(results, key=results.get)
print("Best Model:", best_model_name)

```

Best Model: Gradient Boosting

```

import matplotlib.pyplot as plt
import numpy as np

# Re-create and train Linear Regression and Ridge pipelines for plotting
# The preprocessor, X_train, y_train, X_test, y_test are available from previous cells.
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.pipeline import Pipeline

# Create pipelines for Linear Regression and Ridge
lr_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', LinearRegression())
])
ridge_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', Ridge())
])

# Fit the pipelines
lr_pipeline.fit(X_train, y_train)
ridge_pipeline.fit(X_train, y_train)

# Predictions
y_pred_lr = lr_pipeline.predict(X_test)
y_pred_ridge = ridge_pipeline.predict(X_test)

# Use first 100 points for clarity
n = 100
x = np.arange(n)

plt.figure(figsize=(12,6))

# Plot lines
plt.plot(x, y_test[:n].values,
         color='black', linewidth=2, label='Actual')

plt.plot(x, y_pred_lr[:n],
         color='blue', linestyle='--', linewidth=2, label='Linear Regression')

plt.plot(x, y_pred_ridge[:n],
         color='red', linestyle=':', linewidth=2, label='Ridge Regression')

# Labels
plt.xlabel("Data Points")
plt.ylabel("Insurance Charges")

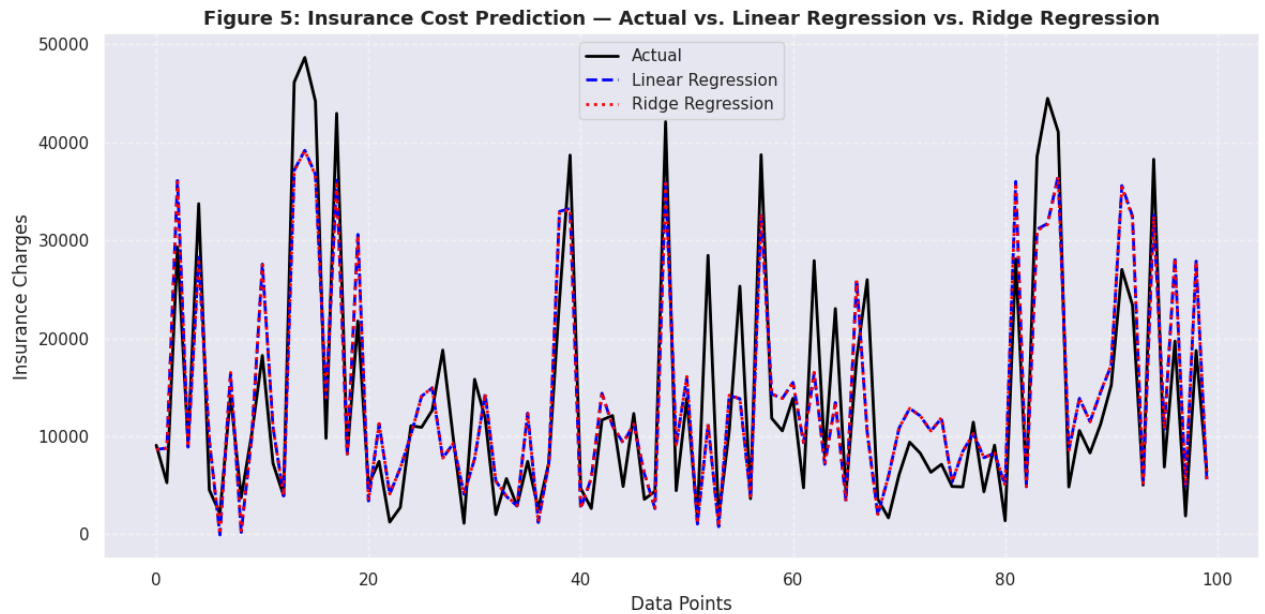
# Title (Figure style)
plt.title("Figure 5: Insurance Cost Prediction – Actual vs. Linear Regression vs. Ridge Regression",
         fontsize=13, fontweight='bold')

```

```
# Legend
plt.legend()

# Grid for clarity
plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```



Hyperparameter Tuning

```
param_grid = {
    'model__n_estimators': [100, 200],
    'model__max_depth': [None, 10, 20]
}

pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor())
])

grid = GridSearchCV(pipeline, param_grid, cv=5, scoring='r2')

grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)

Best Parameters: {'model__max_depth': 10, 'model__n_estimators': 100}
```

Final Model Evaluation

```
best_model = grid.best_estimator_

preds = best_model.predict(X_test)

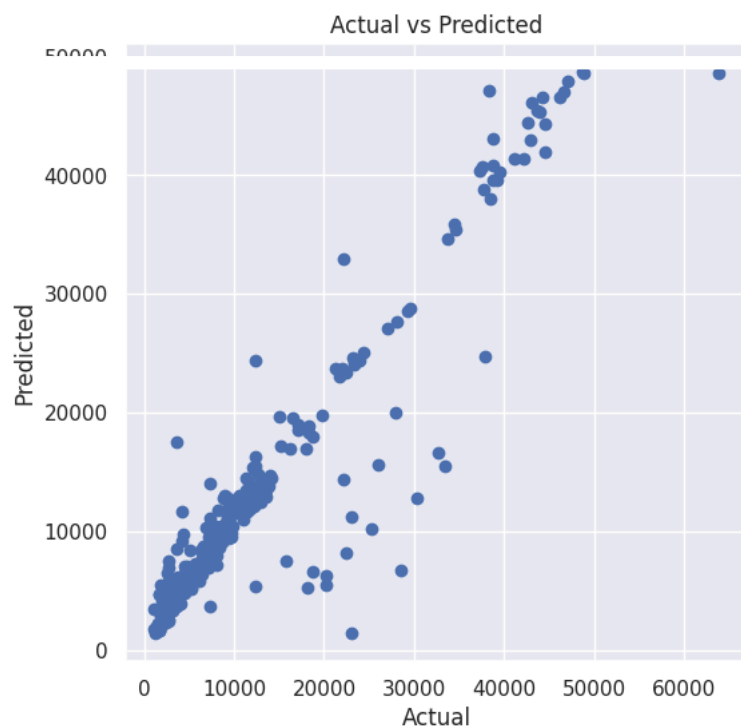
print("Final R2 Score:", r2_score(y_test, preds))
print("Final MSE:", mean_squared_error(y_test, preds))

Final R2 Score: 0.8733646307685287
Final MSE: 19659980.435178295
```

Actual vs Predicted Graph

```
plt.figure(figsize=(6,6))
plt.scatter(y_test, preds)
plt.xlabel("Actual")
```

```
plt.ylabel("Predicted")
plt.title("Actual vs Predicted")
plt.show()
```



Feature Importance

```
model = best_model.named_steps['model']

importances = model.feature_importances_

feature_names = best_model.named_steps['preprocessor']\
    .get_feature_names_out()

plt.figure(figsize=(8,6))
plt.barh(feature_names, importances)
plt.title("Feature Importance")
plt.show()
```

